

H2GCROC + KCU105 Setup and DAQ Manual

Author: Yasuhiro Maruya (yasuhiro.maruya@cern.ch)

change log

- 10-07 Yasuhiro Maruya Writed first version with the inputs from Martin and Norbert.
-
-

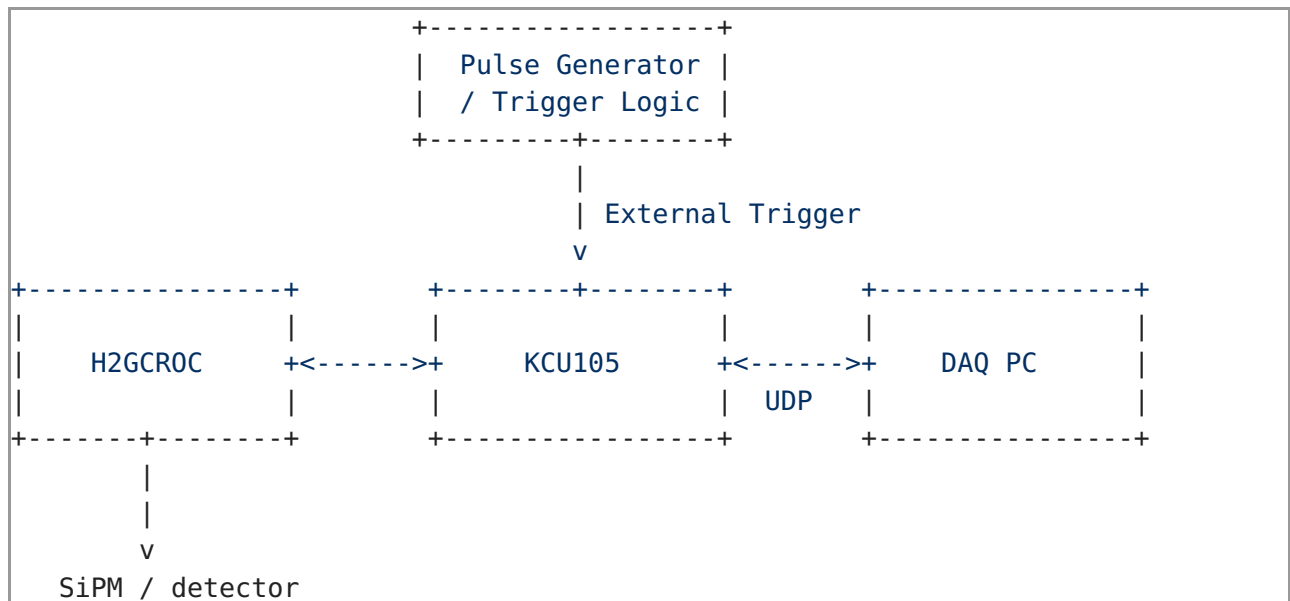
1. Overview

This document describes how to operate the H2GCROC test setup based on a Xilinx KCU105 FPGA board.

The setup currently supports:

- Firmware writing to the KCU105
- Communication with the KCU105
- Configuration of the H2GCROC
- H2GCROC calibration procedures
- Internal trigger operation
- External trigger input
- Data acquisition using RCDAQ
- Online monitoring using pmonitor and roc_analysis

The typical system configuration is:



The KCU105 communicates with the H2GCROC and transfers event data to the DAQ computer through Ethernet/UDP.

The connection can be made using:

- 1 Gbps Ethernet through the RJ45 port

- 10 Gbps Ethernet through the SFP+ port

The 10 Gbps connection from the upper SFP+ slot is recommended for high-rate operation.

2. Hardware

2.1 Required hardware

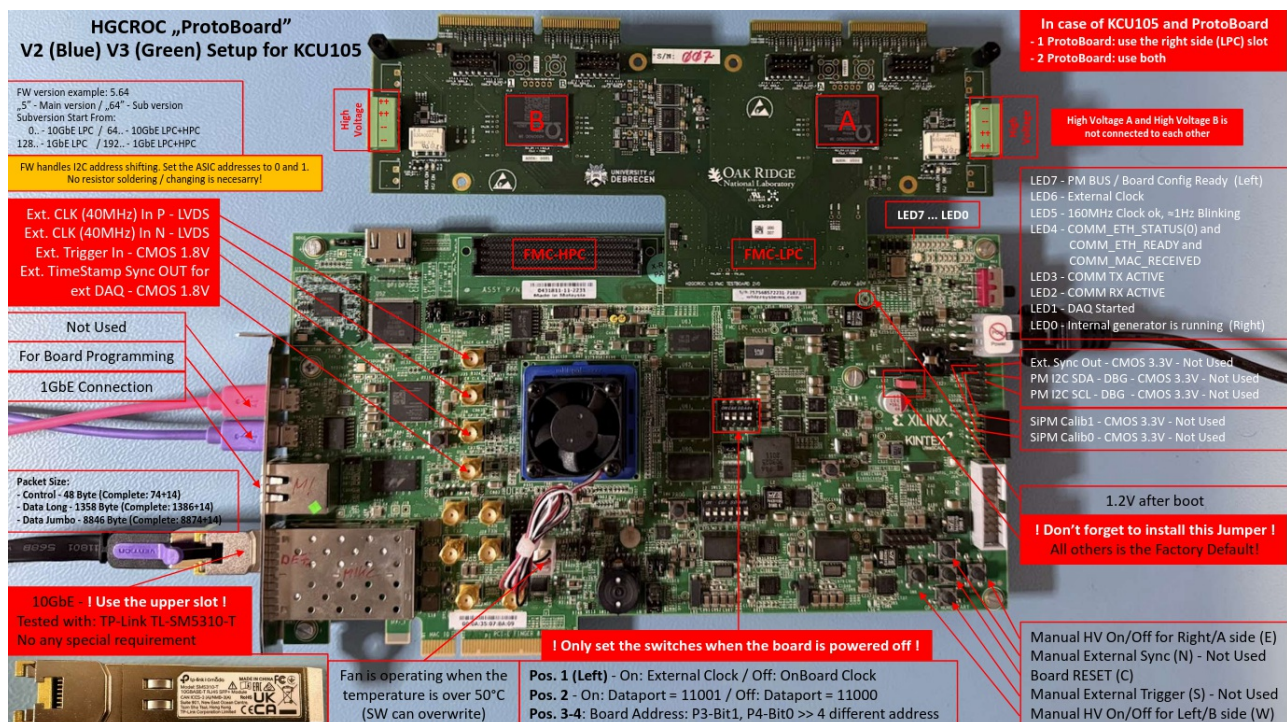
The basic setup consists of:

- Xilinx KCU105 evaluation board
- H2GCROC mezzanine board
 - Several versions exist.
 - The current setup uses the ALICE FoCal-type mezzanine with one H2GCROC ASIC on one carrier board.
- H2GCROC ASIC
- Ethernet connection between the KCU105 and DAQ PC
- Power supplies required by the H2GCROC setup
- External trigger source, if required
- SiPMs and scintillators for detector tests
 - These are not required for basic ASIC calibration.

For basic electronics and DAQ tests, SiPMs are not necessarily required.

2.2 KCU105 connections

Please refer to the following connection diagram:



The main connections are:

Connection	Purpose
Ethernet	Configuration and UDP data transfer: 1 Gbps from RJ45 or 10 Gbps from SFP+
JTAG/USB	FPGA programming and debugging
SMA external trigger input	External trigger; not required for the initial calibration test
H2GCROC mezzanine	ASIC communication and readout; connected to the right-side FMC-LPC connector

Before powering the system, verify all power connections and voltage settings.

3. Software Components

The setup uses several software packages with different purposes.

3.1 H2GCROC calibration software: H2GCalibX

The H2GCROC calibration software is used for:

- I/O delay scan
- Pedestal calibration
- Injection phase calibration
- Injection DAC scan
- TOA calibration
- TOT calibration
- Register configuration
- Internal trigger tests

The calibration software communicates directly with the KCU105.

Repository:

https://gitlab.cern.ch/sjia/h2gcalibx/-/tree/dev-1v5?ref_type=heads

Note: dev - 1v5 is known to work with firmware version 5.202.

3.2 Configuration software: H2GConfig

H2GConfig is used to write an ASIC configuration generated by the calibration software to the H2GCROC.

Software:

<https://cernbox.cern.ch/s/ZH04iOFkIOLCKGi>

3.3 Python DAQ software: H2GDAQ

H2GDAQ is a GUI-based Python DAQ program mainly intended for laboratory tests.

It is used for:

- Taking data with the test setup
- Setting the FPGA generator configuration
- Configuring internal charge injection
- Configuring external trigger operation
- Configuring Machine Gun readout
- Setting the number of ASICs

Repository:

<https://gitlab.cern.ch/sjia/H2GDAQ>

For normal RCDAQ-based data taking, H2GDAQ is mainly useful for configuring the FPGA generator. The actual data acquisition can then be performed by RCDAQ.

3.4 RCDAQ

RCDAQ is used for normal data acquisition.

The main components are:

`rcdaq_server`
`rcdaq_client`

RCDAQ manual:

<https://www.phenix.bnl.gov/~purschke/rcdaq/>

The H2GCROC/KCU105-specific RCDAQ plugin is required to receive and store event packets generated by the KCU105.

Plugin:

https://github.com/mpurschke/EIC_H2GCROC3

3.5 Online monitoring and offline analysis

Online monitoring can be performed using:

`pmonitor`

together with:

`roc_analysis`

RCDAQ/pmonitor manual:

<https://www.phenix.bnl.gov/~purschke/rcdaq/>

H2GCROC analysis package:

https://github.com/mpurschke/roc_analysis

4. FPGA Setup

The correct FPGA firmware must be loaded into the KCU105 before operating the H2GCROC.

Firmware files are available here:

<https://web.tresorit.com/l/86zmA#BE8fq4of3N8pFNz-cXpgg>

The firmware is located in the GGGGGGG directory.

The currently used firmware versions are:

Firmware Network output

5.073 10 Gbps SFP+

5.202 1 Gbps RJ45

10 Gbps is recommended for high-rate operation.

The firmware can be programmed using Vivado Hardware Manager.

Programming instructions:

<https://cernbox.cern.ch/s/0DHcrHhwFolV2f1>

Typical procedure:

1. Connect the KCU105 to the PC through JTAG.
 2. Start Vivado Hardware Manager.
 3. Open the hardware target.
 4. Select the KCU105 FPGA.
 5. Program the FPGA with the appropriate bitstream.
 6. Verify that the FPGA is correctly recognized after programming.
-

5. Network Setup

The KCU105 communicates with the DAQ PC through Ethernet using UDP.

Before starting any calibration or DAQ software, verify that the KCU105 is reachable from the DAQ computer.

For example:

```
ping -s 32 10.1.2.208
```

A successful response confirms basic IP-level communication.

Note that successful ping communication does **not** guarantee that UDP communication is working correctly.

5.1 Firewall

The firewall can block UDP communication even when ping works correctly.

If the software can send commands to the KCU105 but does not receive replies or event packets, check the firewall configuration.

A host firewall has previously caused the H2GCROC software to freeze while waiting for UDP replies.

If necessary:

- allow the required UDP traffic explicitly;
- or temporarily disable the firewall for debugging.

Do not permanently disable the firewall without understanding the network environment.

6. H2GCROC Configuration

After communication with the KCU105 has been established, start the H2GCROC calibration software.

Follow the instructions in the software README.

For the current single-ASIC test setup:

```
number_of_asic = 1
```

Do not use a configuration intended for four ASICs when only one ASIC is connected.

A mismatch in `number_of_asic` can cause incorrect packet handling or software behavior.

6.1 Analog front-end parameters

Several parameters control the H2GCROC analog front end.

Important parameters include:

- `Rf`
- `Cf`
- `Cf_comp`
- `Cd`
- `gain_conv`

`Rf`, `Cf`, and `Cf_comp` have programmable settings from 0 to 15. Some extreme configurations may not provide useful analog behavior.

A commonly used configuration is:

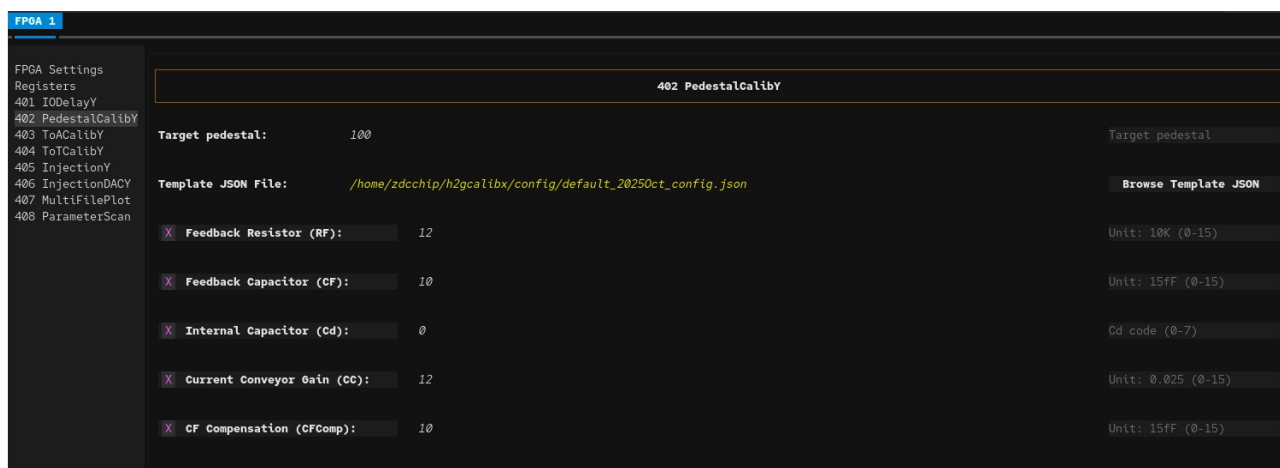
```
Cf      = 1010
Cf_comp = 1010
Rf      = 1100
Cd      = 0
gain_conv = 1100
```

With the detector configuration for which these defaults were developed, the MIP peak is approximately 15 ADC counts.

The optimal configuration depends on:

- SiPM capacitance
- required ADC gain
- required dynamic range
- noise level
- MIP signal size

Therefore, the analog configuration should be validated with an injection linearity scan before detector operation.



7. Basic Calibration Procedure

The following calibration procedures should normally be performed before detector data taking.

A typical calibration sequence is:

```
I0delayY
  ↓
PedestalCalibY
  ↓
InjectionY
  ↓
TOA / TOT calibration
  ↓
InjectionDACy / linearity scan
```

The exact sequence may depend on the firmware and calibration software version.

7.0 IO Delay Scan: I0delayY

I0delayY aligns the digital communication timing between the H2GCROC and the FPGA.

This calibration is fundamentally different from the later injection timing calibration.

Its purpose is to find a stable digital sampling point for communication between the ASIC and FPGA.

An incorrect I/O delay can result in:

- corrupted data;
- missing data;
- unstable register communication;
- apparently random decoding errors.

Therefore, the I/O delay scan should be performed before the other H2GCROC calibration procedures.

Conceptually:

ASIC digital output

|

| scan FPGA sampling delay

v

Find stable timing window

|

v

Choose working IO delay

7.1 Pedestal Calibration: PedestalCalibY

The pedestal calibration determines and equalizes the baseline ADC values.

Two important register parameters are adjusted to move the pedestal to the target ADC value and reduce channel-to-channel variations.

Inv Vref

Inv Vref controls the global pedestal position for each half of the ASIC.

It provides a relatively coarse adjustment shared by a group of channels.

The calibration first adjusts Inv Vref so that the pedestal is close to the requested target ADC value.

Chn Trim Inv

Chn Trim Inv provides a per-channel pedestal correction.

After the global pedestal level has been adjusted with Inv Vref, the individual channels are fine-tuned using Chn Trim Inv.

Conceptually:

```

Target pedestal
  |
  v
Adjust Inv Vref
(global / half-chip)
  |
  v
Adjust Chn Trim Inv
(channel-by-channel)
  |
  v
Uniform pedestal distribution

```

PedestalCalibY performs this pedestal equalization automatically.

After calibration, always verify the resulting pedestal distribution.

Check for:

- channels with abnormal pedestal values;
- channels close to the ADC boundaries;
- channels requiring unusually large trim corrections;
- large residual channel-to-channel variations.

The output configuration generated by this procedure should be used as the input configuration for subsequent calibrations.

7.2 Injection Phase Calibration: InjectionY

InjectionY is used to determine the appropriate **timing phase of the internal charge injection relative to the H2GCROC sampling clock**.

It should not be confused with InjectionDACy.

The distinction is:

```

InjectionY
  -> scan WHEN the charge is injected

InjectionDACy
  -> scan HOW MUCH charge is injected

```

During InjectionY, a fixed calibration charge is injected while the relative injection phase is scanned.

The ADC response changes because the H2GCROC samples the shaped analog pulse at a fixed clock timing.

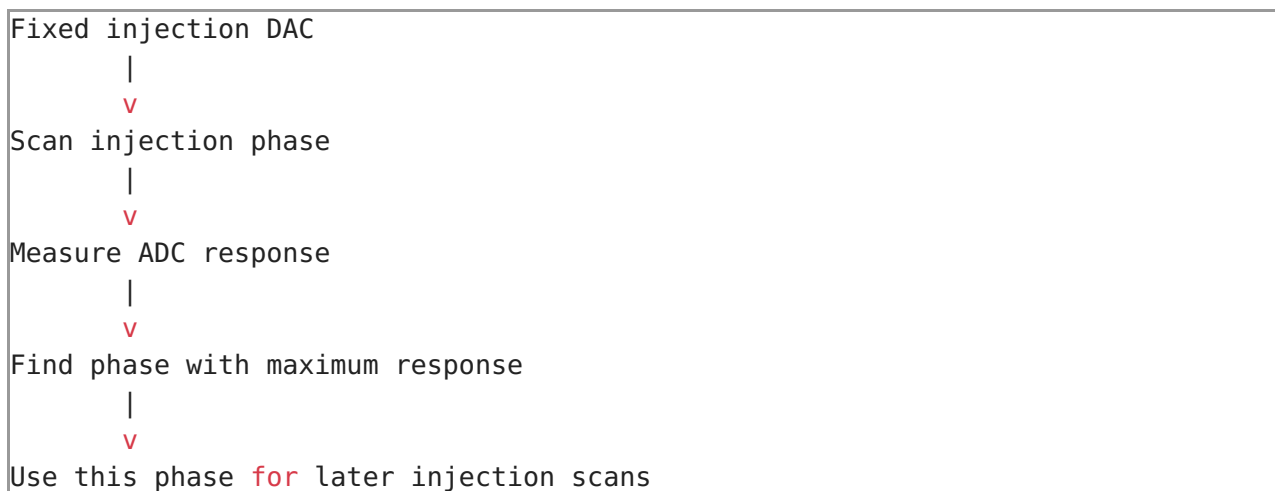
Conceptually:



If the sample is taken too early or too late, the measured ADC value is smaller.

The phase giving the largest ADC response corresponds approximately to sampling near the maximum of the shaped pulse.

The procedure is therefore:



In the present setup, one 25 ns clock period can be divided into fine phase settings. For a 16-step phase scan, the nominal phase spacing is:

$$25 \text{ ns} / 16 = 1.5625 \text{ ns}$$

This **1.56 ns is the injection-phase scan step**, not the intrinsic TOA resolution of the ASIC.

What to check

The result should show a recognizable pulse shape as a function of the injection phase.

Check that:

- the ADC response varies smoothly with phase;
- a clear maximum can be identified;
- all expected phase values are present;
- the selected phase is not unexpectedly located at a scan boundary.

If one phase is systematically missing, for example phase 15, first check the calibration software scan loop and FPGA configuration before interpreting the effect as an ASIC problem.

7.3 TOA and TOT Calibration

H2GCROC provides two timing-related quantities:

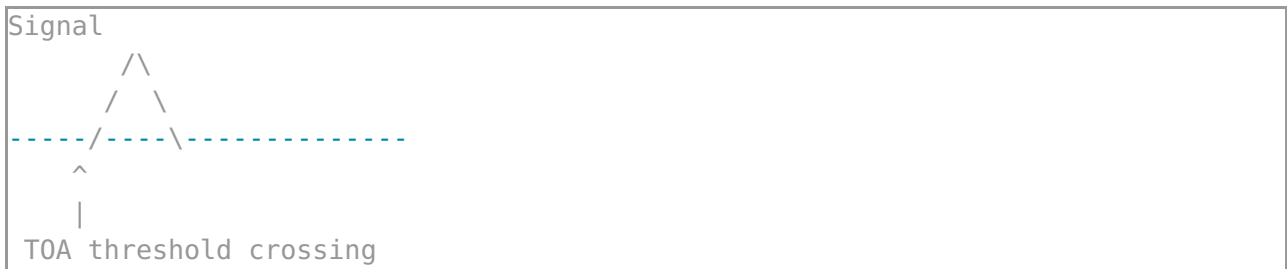
- **TOA — Time of Arrival**
- **TOT — Time over Threshold**

Although both use discriminator/TDC circuitry, they serve different purposes.

7.3.1 TOA: Time of Arrival

TOA measures the timing at which the signal crosses a fast discriminator threshold.

Conceptually:



TOA is therefore mainly a **signal timing measurement**.

The measured time depends on the pulse amplitude because a larger pulse usually crosses the threshold earlier. This effect is known as **time walk**.

The TOA calibration is used to equalize and configure the discriminator thresholds so that the channels respond consistently.

A typical procedure is:

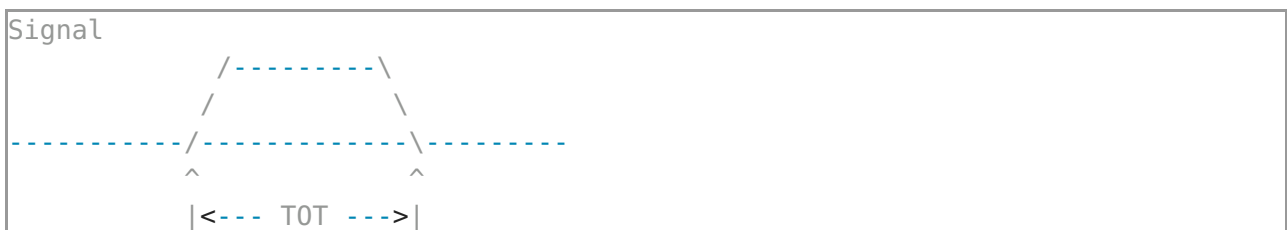
1. Set the global TOA threshold.
2. Inject a fixed charge large enough to cross the TOA threshold.
3. Scan the per-channel TOA threshold trim.
4. Determine an appropriate trim value for each channel.
5. Adjust the global threshold for each chip half if required.
6. Verify that all channels respond consistently.

The TOA calibration therefore primarily calibrates the **TOA discriminator threshold**, rather than the physical TDC time conversion itself.

7.3.2 TOT: Time over Threshold

TOT measures how long the analog signal remains above the TOT discriminator threshold.

Conceptually:



For sufficiently large signals, the H2GCROC preamplifier leaves its normal linear ADC region.

In this region, the duration of the pulse above threshold contains information about the input charge.

TOT therefore provides a charge measurement for larger signals where the normal ADC measurement is no longer sufficient.

The approximate charge-measurement scheme is:

Small / moderate charge

↓
ADC

Large charge / preamplifier saturation

↓
TOT

TOT calibration follows a procedure similar to TOA calibration, but normally uses a larger injected charge so that the signal reliably exceeds the TOT threshold.

A typical procedure is:

1. Configure a sufficiently large injection charge.
2. Set the global TOT threshold.
3. Scan the per-channel threshold trims.
4. Determine the appropriate trim value for each channel.
5. Equalize the response of both chip halves.
6. Verify the TOT response using injected charge.

7.3.3 TOA/TOT thresholds versus Injection DAC

Do not confuse the TOA/TOT threshold DACs with the injection DAC.

They control different quantities:

Injection DAC

-> amount of calibration charge injected

TOA threshold DAC

-> discriminator level for TOA

TOT threshold DAC

-> discriminator level for TOT

The injected charge must simply be large enough to cross the corresponding discriminator threshold during calibration.

7.3.4 Machine Gun

The calibration and DAQ firmware also provides a parameter called Machine Gun.

Machine Gun specifies how many consecutive samples / fast read commands are generated around one trigger.

It is useful for observing the signal over several BX/sample positions.

Conceptually:

```
Trigger
|
+--> sample 0
+--> sample 1
+--> sample 2
+--> sample 3
+--> ...
```

This is especially useful for timing studies and for understanding which sample contains the signal peak.

Machine Gun is a readout configuration parameter; it is not itself an analog calibration parameter.

7.4 Linearity Scan: InjectionDAC

InjectionDAC scans the **internal injection DAC amplitude** while keeping the injection timing approximately fixed.

This is different from InjectionY:

```
InjectionY:
    fixed charge
    scan timing / phase

InjectionDAC:
    fixed timing
    scan injection DAC amplitude
```

The purpose of InjectionDAC is to measure the response of the H2GCROC as a function of injected charge.

Conceptually:

```
Injection DAC
|
| increase
v

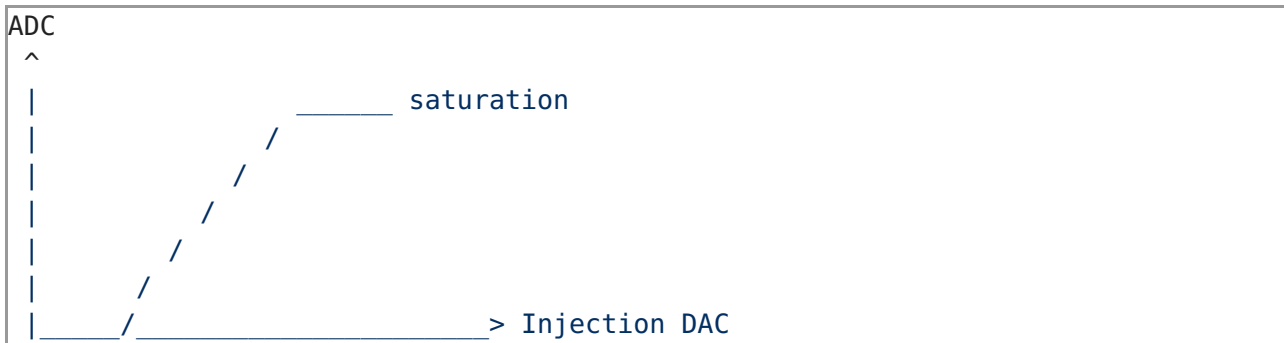
small charge
  ↓
ADC response
  ↓
linear region
  ↓
ADC saturation / transition region
  ↓
TOT region
```

The scan can be used to determine:

- ADC gain;
- ADC linearity;
- usable ADC dynamic range;
- saturation point;
- dependence on R_f , C_f , and other analog settings;
- TOA response versus charge;
- TOT response versus charge.

For each DAC setting, multiple events should be collected and the average or fitted ADC response should be evaluated.

A typical ADC linearity plot is:



The linear region can be fitted with:

$$\text{ADC} = \text{pedestal} + \text{gain} \times \text{InjectionDAC}$$

where gain is expressed in ADC counts per injection-DAC unit.

Important

The raw injection DAC value should **not automatically be interpreted as physical charge in fC or pC**.

A conversion from injection DAC to physical charge requires the calibration of the internal injection circuit.

Therefore:

$$\text{Injection DAC code} \neq \text{physical charge}$$

unless a validated DAC-to-charge conversion is available.

When comparing different analog settings such as R_f and C_f , use the same injection configuration and phase so that the measured differences come from the analog front end rather than timing misalignment.

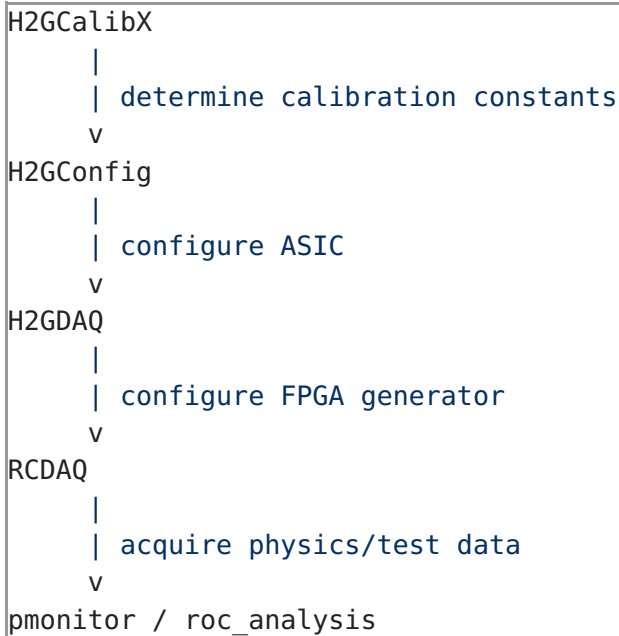
8. Running the DAQ with a Custom Configuration

To run RCDAQ with an optimized H2GCROC configuration:

0. Run I0delayY **using** H2GCalibX.
1. Generate/calibrate the ASIC **configuration using** H2GCalibX.
2. **Write** the ASIC **configuration using** H2GConfig.
3. **Write** the FPGA generator **configuration using** H2GDAQ.
4. Run RCDAQ.

Step 3 may not be required if the nominal FPGA generator configuration is already appropriate.

The three software packages therefore have different roles:



9. H2GConfig Setup

Follow the H2GConfig README.

Typical procedure:

1. Install H2GConfig.
2. Set the path to the desired configuration file or load it from the GUI.
3. Write the configuration to the ASIC.

The configuration should normally be the output of the H2GCalibX calibration sequence.

10. H2GDAQ Setup

Follow the H2GDAQ README.

The generator configuration contains several groups of parameters:

Generator

Debug

Data Collection

Data Coll En[7:0]

1

↕

Trig Coll En[7:0]

0

↕

Jumbo Enable

☐

Generator

Gen PreImp En

☐

Gen Pre Int

15

↕

Gen Nr Cycles

4

Gen Interval

1000

External Trigger

Ext Trg En

☒

Ext Trg Delay

0

↕

Ext Trg DeadT

1000

Fast Command

DAQ FCMD

75

↕

DAQ Push FCMD

75

↕

Gen Pre FCMD

45

↕

Gen FCMD

75

↕

Machine Gun

Machine Gun

20

↕

10.1 Data Collection

Data Coll

Controls which ASIC data streams are enabled.

For example:

1 ASIC:
Data Coll = 1

For two ASICs:

Data Coll = 3

because both lower bits are enabled.

Trig Coll

Controls trigger-data collection.

TODO: Document the exact trigger-collection bit mapping.

Jumbo Enable

Controls the jumbo-packet-related readout configuration.

TODO: Confirm the exact behavior and whether this option changes packet aggregation or compression.

10.2 Generator

Gen PreInp En

Enables the internal calibration charge injection.

Pre Int

Controls the internal injection timing/phase configuration.

TODO: Confirm the exact unit and mapping to the phase used in InjectionY.

Nr Cycles

Number of generator cycles.

Interval

Interval between generator cycles.

TODO: Document the exact unit.

10.3 External Trigger

Ext Trg En

Enables the external-trigger input.

Trg Delay

Sets the delay applied to the external trigger before readout.

TODO: Document the exact unit.

Dead T

Sets the trigger dead time.

Triggers arriving during this period are ignored.

The dead time must be configured so that the system has enough time to handle the previous event and UDP transmission.

10.4 Fast Command

The generator configuration also defines the fast commands used for DAQ and injection operations.

TODO: Document the recommended fast-command values and when they need to be changed.

10.5 Machine Gun

Machine Gun controls the number of samples/read commands acquired around one trigger.

For example, a larger value allows several consecutive BX samples to be read out.

This can be useful for:

- injection pulse-shape studies;
- timing alignment;
- checking ADC values before and after the signal peak.

Increasing the Machine Gun value also increases the amount of data generated per trigger.

After changing the generator parameters, send the generator configuration to the FPGA.

When RCDAQ is used for acquisition, it is not necessary to start data taking from H2GDAQ.

11. RCDAQ Setup

RCDAQ is used to acquire H2GCROC event data from the KCU105.

The installation procedure is described in the RCDAQ manual.

The H2GCROC plugin is available from:

https://github.com/mpurschke/EIC_H2GCROC3

Install the plugin in the same software environment as RCDAQ.

After installation, with `rcdaq_server` running:

```
rcdaq_client load librcdaqplugin_h2gcroc3.so
```

Check:

```
daq_status -ll
```

The output should include:

```
List of loaded Plugins:
- H2GCROC3 Plugin, provides -
- device_h2gcroc3
- device_h2gcroc3_10G
```

The plugin interface is:

```
device_h2gcroc3
(evttype, subid, IP addr, trigger, nr_packets)

device_h2gcroc3_10G
(evttype, subid, IP addr, trigger, nr_packets)
```

For online monitoring, install `online_distribution`.

Use the ePIC branch:

```
git clone -b ePIC git@github.com:SPHENIX-Collaboration/online_distribution.git
```

After installation:

```
dlist -T
```

can be used to test the installation.

11.1 Start the RCDAQ server

Start:

```
rcdaq_server &
```

Verify that the server starts without errors.

11.2 Configure RCDAQ

Load the plugin:

```
rcdaq_client load librcdaqplugin_h2gcroc3.so
```

Add the H2GCROC device to the readout list:

```
rcdaq_client create_device device_h2gcroc3_10G 1 12001 10.1.2.208 1 200
```

The final argument corresponds to the plugin parameter called:

```
nr_packets
```

Todo

The nr is related to the # of events?
Its exact relationship to the H2GCROC event packet structure should be documented more carefully.

Note: In the current setup, device_h2gcroc3_10G should also be used when reading data through the 1 Gbps port.

11.3 Open the RCDAQ GUI

Run:

```
rcdaq_control.pl
```

You may need to install Perl/Tk using yum, dnf, or apt.

11.4 Open a data file

If you want to record the data:

```
daq_open
```

or use **Open** from the GUI.

If you only want online monitoring without writing an event file, opening an output file is not required.

11.5 Start data taking

Start the run:

```
daq_begin
```

or use the corresponding button in the GUI.

Make sure that events are arriving.

For low-level network debugging, tcpdump can be useful to verify UDP traffic.

12. External Trigger Operation

An external trigger can be supplied to the KCU105 through the external-trigger SMA input.

The tested setup is:



The trigger path has been successfully tested with a pulse generator.

12.1 Electrical level

The relevant SMA connector is labeled for a CMOS-level FPGA input.

The current hardware has been described as a **1.8 V CMOS** input.

However, this should be verified against the KCU105 schematic and FPGA I/O constraints before treating it as a permanent specification.

Do **not** apply a 3.3 V or 5 V trigger signal simply because the signal generator labels it as TTL/CMOS.

Before connecting the pulse generator, check:

- output amplitude;
- DC offset;
- termination mode;
- pulse polarity;
- pulse width;
- FPGA input-voltage limit.

12.2 External-trigger configuration

In H2GDAQ:

Ext Trg En = enabled

must be set.

Configure the trigger delay and dead time as required, and send the generator configuration to the FPGA from H2GDAQ.

After this, RCDAQ can be used to receive data generated by the external trigger.

13. Online Monitoring

If data taking is running correctly, online monitoring can be started.

The current monitoring chain is:

```
RCDAQ
|
v
pmonitor
|
v
roc_analysis
|
v
H2GCROC monitoring histograms
```

Compile roc_analysis:

```
cd roc_analysis
make
```

Start the ROOT/pmonitor environment:

```
bash roc1.sh
```

Example:

```
root [0]
Processing roc1.C(0)...
Welcome to pmonitor. Type phelp() for help
```

Connect to the running RCDAQ server:

```
root [1] rcdaqopen()
(int) 0
```

Start processing:

```
root [2] pstart()
(int) 0
```

Create a canvas:

```
root [3] plot(0,2,2)
```

Set automatic updates:

```
root [4] pupdate(c,1)
```

where 1 corresponds to an update period of approximately 1 second.

13.1 Offline monitoring

The same analysis can be run on an existing .evt file.

For example:

```
bash roc1.sh <path-to-data>/<file>.evt
```

Example ROOT session:

```
Processing roc1.C("/data/phnxrc/BIC/h2gcroc_testfile.evt")...
Welcome to pmonitor. Type phelp() for help

root [1] prun()
root [2] plot(0,5,5)
```

14. Summary of the Complete Setup

The H2GCROC + KCU105 system can be understood as four separate layers:

Layer	Software / Hardware	Main role
ASIC	H2GCROC	Amplification, ADC, TOA, TOT, trigger-related processing
Front-end control	KCU105 FPGA	ASIC control, trigger handling, charge injection, packet generation
Calibration/configuration	H2GCalibX / H2GConfig / H2GDAQ	Determine and write ASIC/FPGA settings
Data acquisition	RCDAQ + roc_analysis	Receive, record, decode, and monitor events

The main calibration quantities are:

Calibration	Scanned quantity	Main purpose
I0delayY	FPGA/ASIC digital timing	Establish reliable digital communication
PedestalCalibY	Inv Vref + Chn Trim Inv	Equalize ADC pedestal
InjectionY	Injection timing/phase	Find appropriate sampling phase
TOA calibration	TOA discriminator settings	Equalize TOA threshold response
TOT calibration	TOT discriminator settings	Equalize TOT threshold response
InjectionDAC	Injection DAC amplitude	Measure gain, linearity, saturation, and dynamic range

The important distinction is:

IOdelayY -> digital communication timing InjectionY -> analog injection/sampling timing InjectionDACY -> injected charge amplitude TOA -> signal arrival timing TOT -> signal duration above threshold / large-signal charge information
--

14.1 Recommended calibration sequence

Program FPGA ↓ Check network ↓ IOdelayY ↓ PedestalCalibY ↓ InjectionY ↓ TOA calibration ↓ TOT calibration ↓ InjectionDACY / linearity check ↓ Save calibrated ASIC configuration

14.2 Recommended DAQ startup sequence

```

Program KCU105
  ↓
Check ping / UDP connectivity
  ↓
**Do the I0delay Scan**
  ↓
Configure H2GCROC with H2GConfig
  ↓
Configure FPGA generator with H2GDAQ
  ↓
Start rcdaq_server
  ↓
Load H2GCROC RCDAQ plugin
  ↓
create_device
  ↓
daq_open
  ↓
daq_begin
  ↓
rcdaqopen() + pstart()
  ↓
Check online monitoring

```

14.3 Minimal validation without SiPMs

Before connecting SiPMs, the following chain can already be tested:

```

Internal charge injection
  +
External trigger
  ↓
KCU105
  ↓
H2GCROC
  ↓
UDP packets
  ↓
RCDAQ
  ↓
pmonitor / roc_analysis

```

Successful data taking with this configuration verifies most of the readout chain independently of the detector.

The next step is then:

SiPM + scintillator

↓

H2GCROC

↓

KCU105

↓

RCDAQ

↓

Online monitoring

15. Troubleshooting Guide

15.1 KCU105 cannot be pinged

Check:

```
[ ] Ethernet cable
[ ] Correct Ethernet interface
[ ] PC IP address
[ ] KCU105 IP address
[ ] Subnet configuration
[ ] Correct firmware loaded
```

15.2 ping works but H2GCalibX/H2GDAQ does not receive data

The most likely problem is UDP communication.

Check:

```
[ ] Firewall
[ ] UDP ports
[ ] Network interface
[ ] Correct KCU105 IP
[ ] Correct firmware
```

A firewall can allow ICMP ping while blocking the UDP packets required by H2GCROC software.

15.3 Calibration software freezes while waiting for a reply

Check the firewall first.

Use a packet monitor if necessary:

```
tcpdump
```

If outgoing packets are visible but replies never reach the application, check:

- firewall;
- UDP port configuration;

- interface routing.
-

15.4 Software reports four ASICs but only one ASIC is connected

Set:

`number_of_asic = 1`

The number of ASICs in the software configuration must match the actual hardware.

15.5 Pedestal calibration gives abnormal channels

Check:

- Inv Vref;
- Chn Trim Inv;
- correct ASIC configuration;
- analog front-end configuration;
- whether the channel is saturated at the ADC boundary.

Repeat PedestalCalibY after correcting the configuration.

15.6 InjectionY does not show a clear maximum

Possible causes include:

- incorrect injection DAC;
- injection disabled;
- incorrect injection timing range;
- incorrect Machine Gun configuration;
- wrong ADC sample being inspected;
- analog configuration with too little gain.

First verify that the charge injection itself is working.

Then check the full phase scan rather than only one phase.

15.7 One InjectionY phase is missing

If a phase such as phase 15 is systematically absent:

1. Check whether the software loop actually requests all phase values.
2. Check whether the FPGA accepts the final phase setting.
3. Compare with a neighboring phase.
4. Repeat the scan.
5. Check whether the issue depends on firmware/software version.

Do not immediately interpret a missing phase as an ASIC analog problem.

15.8 Injection DAC scan is nonlinear

Check the H2GCROC analog settings:

Rf
Cf
Cf_comp
Cd
gain_conv

Different front-end configurations can have different gain and saturation behavior.

Also check whether the ADC has already reached its expected nonlinear/saturation region.

A large injected charge may need to be evaluated using TOT rather than ADC.

15.9 TOA values are missing

Check:

- TOA discriminator threshold;
- injected charge;
- TOA trim calibration;
- injection timing;
- whether the signal actually exceeds the TOA threshold.

A charge that produces a visible ADC signal does not necessarily guarantee that the TOA discriminator threshold is crossed.

15.10 TOT values are missing

Check:

- TOT threshold;
- injected charge;
- TOT trim;
- analog configuration.

TOT generally requires a significantly larger signal than a simple ADC measurement.

15.11 Internal trigger works but external trigger does not

Check:

```
[ ] Correct SMA connector
[ ] Ext Trg En enabled
[ ] Correct pulse amplitude
[ ] Correct DC offset
[ ] Correct pulse polarity
[ ] Correct pulse width
[ ] Trigger delay
[ ] Trigger dead time
```

Use an oscilloscope to confirm the actual voltage at the KCU105 input.

15.12 External trigger reaches KCU105 but RCDAQ records no events

Check the chain sequentially:

```
External trigger
    ↓
FPGA accepts trigger?
    ↓
H2GCROC readout generated?
    ↓
UDP packet transmitted?
    ↓
DAQ PC receives UDP?
    ↓
RCDAQ plugin receives packet?
```

Use tcpdump to separate a network problem from an RCDAQ problem.

15.13 RCDAQ event rate is much lower than trigger-generator rate

Check:

- nr_packets;
- Machine Gun setting;
- external-trigger dead time;
- UDP packet loss;
- packet/event structure;
- RCDAQ device configuration.

The trigger frequency should not automatically be expected to equal the RCDAQ event frequency until the packet structure and RCDAQ event-building behavior are understood.

16. Troubleshooting Log

This section records problems actually encountered during setup development.

Add new entries whenever a new issue is understood. Recording both the symptom and the final cause is particularly useful for future users.

Date	Symptom	Cause / Diagnosis	Solution / Status
2026-09-24	UDP communication froze even though the KCU105 could be contacted	Host firewall, including Norton, blocked UDP communication	Allow the required UDP traffic or temporarily disable the firewall for debugging
2026-09-25	One expected phase, including phase 15 in some InjectionY tests, was not observed	Not yet fully identified; possible software/firmware phase-scan issue	Repeat scan and check requested phase values and firmware behavior
2026-09-25	Some Rf/Cf combinations showed poor ADC linearity	Analog front-end response depends strongly on the selected configuration	Compare configurations with the same InjectionDAC scan and select a configuration with sufficient gain and dynamic range
2026-10-07	External-trigger events were successfully received from the KCU105	External-trigger path confirmed operational	External trigger can be used for RCDAQ tests
2026-10-07	Pulse generator was set to 100 Hz, but recorded rate was about 3 Hz for <code>nr_packets = 200</code> and about 30 Hz for <code>nr_packets = 20</code>	Event/packet handling depends strongly on the RCDAQ <code>nr_packets</code> parameter	Exact event-building relationship should still be documented; do not interpret the difference as trigger inefficiency
2026-10-07	RCDAQ data could be written with <code>daq_open</code> and monitored with <code>pmonitor/roc_analysis</code>	Full KCU105 → UDP → RCDAQ → monitoring chain confirmed	Basic DAQ chain is operational

16.1 Open issues to be documented

The following items should still be clarified or documented:

- Exact meaning of `nr_packets` in terms of H2GCROC packet/event structure
- Recommended `nr_packets` value for each Machine Gun configuration
- Exact external-trigger electrical specification
- Exact `Trg_Delay` unit
- Exact `Dead_T` unit
- Exact generator `Interval` unit
- Exact meaning of `Trig_Coll`
- Exact behavior of `Jumbo_Enable`
- Mapping of `Pre_Int` to the injection phase used by H2GCalibX
- Recommended `Rf`, `Cf`, `Cf_comp`, `Cd`, and `gain_conv` settings
- Injection DAC to physical charge calibration
- Recommended SiPM operating configuration
- Expected event size for each readout configuration

17. Quick Reference

For calibration:

```
FPGA firmware
  ↓
Network check
  ↓
I0delayY
  ↓
PedestalCalibY
  ↓
InjectionY
  ↓
T0A / T0T calibration
  ↓
InjectionDACY
  ↓
Save configuration
```

For normal data taking:

```
H2GConfig
  ↓
H2GDAQ generator configuration
  ↓
rcdaq_server
  ↓
load H2GCROC plugin
  ↓
create_device
  ↓
daq_open
  ↓
daq_begin
  ↓
pmonitor / roc_analysis
```

The key idea is to keep the roles separated:

```
H2GCalibX = determine calibration parameters
H2GConfig = write ASIC configuration
H2GDAQ    = configure FPGA generator
RCDAQ     = acquire and store event data
roc_analysis = decode and monitor event data
```